



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): René Adrián Dávila Pérez

Asignatura: Programación Orientada a Objetos

Grupo: 07

No de Práctica(s): 2

Integrante(s): 323331229 323195386

426060019 321321976

323217156

312109239

No. de lista o brigada: 6

Semestre: 3^{ro}

Fecha de entrega: 04/09/2026

Observaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	3
1.1. Planteamiento del problema	3
1.2. Motivación	3
1.3. Objetivos	3
2. Marco Teórico	4
3. Desarrollo	5
3.1. Ejercicio 1	5
3.2. Ejercicio 2	6
3.3. Ejercicio 3	6
4. Resultados	7
4.1. Resultados del Ejercicio 1	7
4.2. Resultados del Ejercicio 2	8
4.3. Resultados del Ejercicio 3	8
5. Conclusiones	9

1. Introducción

1.1. Planteamiento del problema

En esta práctica el problema consta de tres partes. La primera consiste en preguntar a el LLM cómo funciona el código el cual es capaz de generar una relación de alumnos aprobados y reprobados que incluye un criterio variable el cual es un bono (compensación) para el maestro. La segunda consiste en ingresar una serie de 10 números enteros de donde se determina e imprime el valor entero más grande con una restricción de usar solo tres variables específicas (contador, numero, mayor). La tercera consistía en implementar el uso correcto de al menos un ciclo para imprimir una tabla de valores.

1.2. Motivación

Esta práctica busca que aprendamos a apoyarnos en la inteligencia artificial para entender códigos complejos, además nos obliga a recordar fundamentos de la lógica computacional mediante la resolución de algoritmos con restricciones estrictas como lo sería solo usar tres variables en el ejercicio 2 y el uso de estructuras de control iterativas en el ejercicio 3.

1.3. Objetivos

- **General:** Desarrollar y reforzar las capacidades de lógica computacional mediante la resolución de algoritmos sujetos a restricciones. Con la integración del uso de LLM como herramienta de apoyo para el análisis de código.
- **Específicos:**
 - Analizar el funcionamiento de un código para la gestión de alumnos aprobados y reprobados con un comportamiento de bonificación, utilizando nuestro LLM para comprobar que cumple los casos establecidos.
 - Ingresar una serie de 10 enteros, de donde se determine e imprima el valor entero más grande, solo usando 3 variables: contador, número, mayor
 - Imprimir una tabla de valores usando al menos un ciclo.
 - Implementar estructuras de control de flujo (if/else, switch, for, while, etc.).

2. Marco Teórico

1. Tipos de dato: Se refieren al tipo de información que se maneja, en donde la unidad mínima de almacenamiento es el dato. Define un conjunto de valores y las operaciones sobre estos valores. [3]
2. Tipos de dato primitivos: Datos sencillos que contienen los tipos de información más habituales: valores booleanos, caracteres y valores numéricos enteros o de punto flotante [3]
3. Tipos de dato referencia: representan datos compuestos o estructuras, es decir, referencias a objetos. Estos tipos de dato almacenan las direcciones de memoria y no el valor en sí (similares a los apuntadores en C). [3]
4. Estructuras de control: Permiten cambiar el flujo de ejecución de las instrucciones de un programa. En otras palabras permiten tomar decisiones o realizar un proceso repetidas veces. [2]
5. Operadores aritméticos: Son operadores binarios (requieren siempre dos operandos) que realizan las operaciones aritméticas habituales: suma (+), resta (-), multiplicación (*), división (/) y resto de la división o módulo. [3]
6. Operadores de asignación: Los operadores de asignación permiten asignar un valor a una variable. El operador de asignación por excelencia es el operador igual (=). [2]
7. Operadores incrementales: Java dispone del operador incremento (++) y decremento (--). [2]
8. Operadores relacionales: Sirven para realizar comparaciones de igualdad, desigualdad y relación de menor o mayor. El resultado de estos operadores es siempre un valor boolean (true o false) según se cumpla o no la relación considerada. [4]
9. Estructuras de repetición: Las estructuras de repetición, lazos, ciclos o bucles se utilizan para realizar un proceso repetidas veces. El código incluido entre las llaves (opcionales si el proceso repetitivo consta de una sola línea), se ejecutará mientras se cumplan determinadas condiciones. [3]
10. Integer.MIN_VALUE: Es una constante en la clase Integer del paquete java.lang que especifica que almacena el valor máximo posible para cualquier variable entera en Java. [1] [4]

3. Desarrollo

3.1. Ejercicio 1

El trabajo para este ejercicio se centró en colaborar con nuestro LLM preguntándole sobre el funcionamiento del código que fue proyectado en clase, el cual funciona de la siguiente manera:

- Es capaz de establecer una relación. de alumnos aprobados y reprobados que incluye un criterio variable que es un bono (compensación) para el maestro, asegurando que el código cumpla con cuatro casos, en el primero todos aprueban y hay una salida especial en la que se menciona que el maestro recibe un bono, en el segundo uno de los 10 reprueba pero aún sigue apareciendo el bono para el maestro, en el tercero 2 reprueban y el bono deja de aparecer como en el caso de la foto y por último caso en el que todos reprueban
- Se le pregunto al LLM lo siguiente: Con esta imagen dime cómo funciona el código. Considera que hay cuatro casos en el primero todos aprueban y en hay una salida especial en la cual se menciona que el maestro recibe un bono, en el segundo uno de los 10 reprueba, pero aún sigue apareciendo el bono para el maestro, en el tercero 2 reprueban y el bono deja de aparecer como en el caso de la foto y por último caso en el que todos reprueban

A partir del análisis realizado con el LLM, se identificó el funcionamiento del programa:

- El programa comienza declarando dos variables enteras inicializadas en cero. Estas variables actúan como contadores, una para llevar el registro de los aprobados y otra para los reprobados.
- El código utiliza un ciclo iterativo (como un for o un while) configurado para repetirse exactamente 10 veces. En cada iteración de este ciclo:
- Se imprime en la terminal el mensaje: Ingresa el resultado (1 = aprobó, 2 = reprobó).
- El programa pausa su ejecución esperando a que el usuario introduzca un número entero por teclado.
- Inmediatamente después de capturar el número ingresado, hay una estructura condicional if-else dentro del mismo ciclo:
- Si el número es 1: El contador de aprobados se incrementa en 1 (aprobados++).
- Si el número es 2: El contador de reprobados se incrementa en 1 (reprobados++).
- Una vez que el ciclo termina sus 10 vueltas, el programa imprime el valor final de los contadores.
- Finalmente, entra en juego la regla especial del bono. Según los casos (10 y 9 aprobados dan bono, pero 8 o menos ya no), el programa tiene un condicional final fuera del ciclo. La condición lógica que determina esto es que los aprobados sean mayores o iguales a 9

3.2. Ejercicio 2

Para este ejercicio se debía ingresar una serie de 10 números enteros y determinar el valor más grande, con la restricción de usar únicamente tres variables: contador, número y mayor por lo que decidimos usar la siguiente lógica.

- Se declaran variables de tipo entero. contador (para controlar las repeticiones) inicia en 0, mayor (para almacenar el número más grande encontrado) inicia en 0, y numero (para guardar temporalmente lo que teclea el usuario).
- Se inicia un ciclo que se repetirá mientras el contador sea menor que 10. Al empezar en 0, esto significa que el ciclo se ejecutará exactamente 10 veces.
- En cada repetición, el programa le pide al usuario que ingrese un valor (Imprimir "ingresar número") y guarda ese valor en la variable número (leer número).
- Evalúa si el número que el usuario acaba de ingresar es más grande que el que está guardado en mayor. Si la condición se cumple, actualiza la variable mayor para que tome ese nuevo valor.
- Se le suma 1 a la variable contador (contador++).
- Una vez que el contador llega a 10, el ciclo Mientras termina. El algoritmo pasa a la última instrucción, que es imprimir el valor final de la variable mayor, mostrando así el número más alto ingresado, y luego termina (FIN).

Como prueba, se ingresaron 10 números, verificando en todos los casos que la variable mayor se actualizaba correctamente y que el programa respetaba la restricción de usar únicamente las tres variables permitidas.

3.3. Ejercicio 3

En este ejercicio lo que se requería era imprimir la tabla de valores presentada en clase haciendo uso de ciclos. En este caso usamos la siguiente lógica.

- Antes de realizar cualquier cálculo, imprime una sola línea de texto en la pantalla con los títulos de las columnas ("n, 10*n, 100*n, 1000*n").
- Inicia un bucle que le indica a la computadora que repita un bloque de instrucciones. La variable n arranca con el valor de 1 y el ciclo se repetirá hasta que n llegue a 5, incrementando de uno en uno.
- En cada iteración, el programa toma el valor actual de n y lo multiplica por 10, 100 y 1000. Los resultados se guardan en las variables correspondientes.
- Después de hacer los cálculos, el programa imprime en una misma línea el valor de n y los tres resultados obtenidos, separándolos con espacios en blanco para intentar alinearlos debajo de los encabezados de la tabla.

Como prueba, se verificó que la tabla generada mostrara los cinco renglones esperados (n de 1 a 5) con los valores correctos de $n \times 10$, $n \times 100$ y $n \times 1000$ en cada columna.

4. Resultados

4.1. Resultados del Ejercicio 1

Se muestra la respuesta obtenida al consultar al LLM sobre el funcionamiento del código.



Figura 1: Respuesta del LLM.

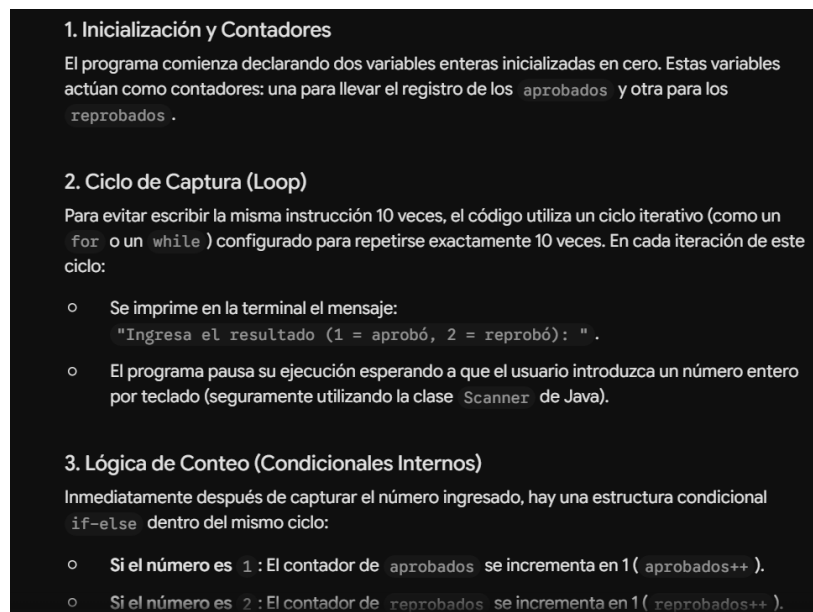


Figura 2: Respuesta del LLM.

4.2. Resultados del Ejercicio 2

Se muestra la ejecución del programa al ingresar los 10 números solicitados por teclado.

```
PS C:\Users\o-ale> cd Downloads
PS C:\Users\o-ale\Downloads> java Ejercicio_2
Bienvenido al Detector del mayor de 10 números
Ingrese el numero 1: █
```

Figura 3: Ingreso de números .

Una vez capturados los 10 valores, el programa imprime el número más grande encontrado.

```
Bienvenido al Detector del mayor de 10 números
Ingrese el numero 1: 34
Ingrese el numero 2: 56
Ingrese el numero 3: 12
Ingrese el numero 4: 23
Ingrese el numero 5: -55
Ingrese el numero 6: 23
Ingrese el numero 7: 255
Ingrese el numero 8: 117
Ingrese el numero 9: 1906
Ingrese el numero 10: 2026
El mayor de los numeros ingresados es: 2026
PS C:\Users\o-ale\Downloads> █
```

Figura 4: Salida del código .

4.3. Resultados del Ejercicio 3

Se muestra la tabla de valores generada por el programa, con los encabezados y los cinco renglones esperados.

```
PS C:\Users\o-ale\Downloads> java Ejercicio_3
n      |  n * 10  |  n * 100  |  n * 1000
-----|-----|-----|-----
1      |    10    |    100    |   1000
2      |    20    |    200    |   2000
3      |    30    |    300    |   3000
4      |    40    |    400    |   4000
5      |    50    |    500    |   5000
PS C:\Users\o-ale\Downloads> █
```

Figura 5: Salida del código .

5. Conclusiones

El análisis de los resultados obtenidos demuestra que la ejecución en Java depende de un buen manejo de estructuras de control iterativas y condicionales, como se hizo en el proceso de evaluación de alumnos, el uso de condiciones (`if-else`) y contadores dentro de un ciclo fue fundamental para ejecutar correctamente los diferentes casos de aprobación y reprobación.

La resolución de la búsqueda del valor máximo al usar solamente tres variables las cuales son: contador, número y mayor se necesitó de las de las asignaciones lógicas. En este caso, el uso de la constante `Integer.MIN_VALUE` garantizó que la primera comparación entre la variable que guarda el número mayor y el primer número del arreglo fuera justa, ya que en el proceso de construcción del código, al introducir números negativos en todos los valores resultaba un cero, sin embargo, tal número nunca fue ingresado, el uso de la asignación lógica permitió el correcto funcionamiento del código tomando en cuenta tal excepción.

La generación de la tabla de los numeros demostró cómo las estructuras de repetición, específicamente el ciclo `for`, operan para automatizar operadores aritméticos sin requerir repitición en las líneas del código. Además del uso de la concatenación para que la tabla se mostrara de manera correcta y definida con el formato solicitado, como los espacios de tabulación entre columnas.

Referencias

- [1] Java. (s.f.). Java. Recuperado 3 de septiembre de 2026, de <https://www.java.com/es/>
- [2] Programador Certificado Java 2. (2.a ed.). (1992). Alfaomega Grupo Editor.
- [3] Introducción a la programación con Java (1.a ed.). (2009). Mc Graw Hill.
- [4] SCJP Sun Certified Programmer for Java 6 Study Guide. (8a. C.). Mc Graw Hill.